That is it. Automatic execution of the test scripts begins, and you will observe that the tests pass, as shown in figure 10.2. The `truffle test` command initiates the execution of the test script, compiles and deploys the contract, and then runs the test script. The response screenshot is shown in figure 10.2.

```
Contract: Counter
  ✓ Success on initialization of counter. (55ms)
  ✓ Success on decrement of counter. (116ms)
  ✓ Success on increment of counter. (132ms)
  ✓ Failure on initialization of counter with negative number. (83ms)
  ✓ Failure on underflow of counter. (71ms)
  ✓ Failure on increment with negative numbers. (65ms)
  ✓ Failure on decrement with negative numbers. (69ms)


7 passing (1s)
```

Figure 10.2 Test outputs for the counter smart contract

You can observe three tests for the three functions, and four tests for the modifiers work as expected, verified by the assertions at the end of the test script functions. The output messages for the positive tests begin with `'Success on'`, and those for negative tests begin with `'Failure on'`. Follow this best practice in your design of test scripts to identify the test type. The check before the test outcomes indicates that the test has passed. The time shown in parentheses indicates the time it took to perform the test. Times greater than 100ms are displayed in red, in case you want to reduce the time and cost of execution of a certain test. These times are a concern when you test this smart contract on a real network such as an Ethereum mainnet. For